

# MUSE: A Multi-Feature Semantic Fusion Method for ROS Node Search Based on Knowledge Graph

Yuxin Zhao<sup>1,2</sup>, Xinjun Mao<sup>1,2\*</sup>, Tanghaoran Zhang<sup>1,2</sup>, Zhang Zhang<sup>1,2</sup>

<sup>1</sup>College of Computer, National University of Defense Technology, Changsha, China

<sup>2</sup>Key Laboratory of Software Engineering for Complex Systems, National University of Defense Technology, Changsha, China  
{yuxinzha, xjmao, zhangthr, zhangzhang14} @nudt.edu.cn

**Abstract**—Reusing ROS components, specifically ROS Nodes, is crucial for improving the efficiency and quality of robotic software development. However, developers face challenges in finding the desired ROS Nodes for reuse due to scattered organization of ROS Nodes and the ambiguity in their ROS Node name. To address these challenges, this paper proposes a Multi-feature SEmantic fusion method (MUSE) that leverages a domain-specific ROS knowledge graph for searching ROS Nodes. Firstly, a large dataset is constructed, comprising code files and textual descriptions related to ROS Nodes obtained from GitHub and ROS Wiki. Secondly, an in-depth analysis of user queries regarding the reuse of ROS Nodes is conducted, leading to the selection of multiple features that provide a comprehensive representation of ROS Node semantics, including *Function*, *Hardware*, *Input*, and *Output*. Subsequently, a knowledge graph of ROS Nodes is developed based on the dataset, incorporating the selected features. This knowledge graph effectively organizes scattered knowledge and resolves the issue of diverse mentions through entity disambiguation and resolution. To eliminate the semantic gap between the descriptions of features mentioned in user queries and the entities in the knowledge graph, a pre-trained transformer-based model was used to measure the multi-feature semantic similarity between user queries and ROS Nodes knowledge. Finally, we employ a linear regression model to integrate the multi-feature knowledge between user queries and ROS Nodes knowledge. The proposed method has shown a 20% improvement in performance on NDCG@1 compared to other ROS Node search methods. Further evaluations highlight the effectiveness of each feature incorporated in the knowledge graph, as well as the significance of each parameter within the regression model. These findings underscore the robustness of this research in optimizing the reuse of ROS Nodes and facilitating the development of robotics software.

**Index Terms**—Knowledge Graph, ROS Nodes, Robotics software development

## I. INTRODUCTION

The field of robot software development has witnessed significant progress, with the Robot Operating System (ROS) [24] emerging as the most widely adopted platform. ROS Nodes serve as a fundamental component throughout the process of robot software development, and developers can leverage ROS Nodes to construct complex and sophisticated robot software systems. The ROS community and GitHub offer a wealth of open-source ROS Nodes with a diverse range of functionalities. However, when searching for nodes in ROS Wiki, developers can only obtain basic information about the nodes. To access more detailed information and extension details

about ROS Nodes, developers must further research resources such as GitHub. By searching existing ROS Nodes from the ROS community and GitHub, developers can leverage and reuse ROS Node components to enhance their development efficiency and software quality.

However, existing search methods [7], [15] face numerous challenges when striving to meet user requirements with high accuracy due to scattered ROS Node knowledge, and the gap in both ROS and user query semantics. For the first challenge, scattered ROS Node knowledge arises from the absence of a universal standard or website for organizing and categorizing ROS Nodes, resulting in diverse contributions by scholars and institutions [31]. For the second challenge, ROS Nodes are executable code files, so it is challenging for describing the characteristic of a ROS Node accurately, clearly, and concisely using natural language. Consequently, the rich semantics of the code in ROS Nodes pose difficulties when searching for relevant ROS Nodes. Moreover, the textual information of ROS Nodes frequently contains domain-specific vocabulary, creating a semantics gap between ROS Node knowledge and users' query [11].

Based on the aforementioned challenges in searching ROS Nodes, we propose MUSE, a multi-feature semantic fusion method for this task based on a domain-specific knowledge graph. To support knowledge graph building and ROS Node searching, we construct a large dataset consisting of code files and textual information from ROS Wiki and GitHub. This large dataset combines the textual information from ROS Wiki and the code information from GitHub, providing a more comprehensive understanding of ROS Nodes. To provide an accurate, clear, and concise representation of ROS Node information, multi-features including *Function*, *Hardware*, *Input*, and *Output* are identified based on an analysis of user queries of ROS Node. Subsequently, a ROS Node knowledge graph is built by extracting and integrating the above features to effectively portray ROS Nodes. Moreover, we use pre-trained transformer-based models to measure the similarity between user queries and feature descriptions to eliminate their semantic gap. Finally, we incorporate the similarity across all features within a linear regression model, aiming to enhance the accuracy of ROS Node search.

The specific contributions of this paper can be summarized as follows:

- We construct a ROS domain dataset containing 7,172 code files and their related text description, thereby offering valuable data support for future research on ROS Nodes. A comprehensive ROS Node knowledge graph is also built to organize the scattered ROS knowledge effectively by four representative features, which is identified from an in-depth analysis of user queries.
- We propose a multi-feature semantic fusion method that leverages ROS corpus and pre-trained transformer-based models to accurately calculate the similarity between ROS Nodes and user queries. Integration of the contributions from multiple features is achieved through a linear regression model.
- Experiment results demonstrate that the proposed method has shown a 20% improvement in performance on NDCG@1 compared with other ROS Node search methods. Moreover, individual research findings highlight the significance and effectiveness of each feature and parameter employed in our approach.

The remainder of this paper is organized as follows. Section II introduces related work. Section III describes our methodology. Experimental settings are provided in Section IV while results are illustrated in Section V. Finally, we conclude the paper in Section VI.

## II. RELATED WORK

In this section, we review related studies pertaining to ROS component reuse and ROS Node reuse.

### A. ROS Component Reuse

ROS plays a crucial role in robotics software development by offering a wide range of reusable components, including ROS Packages, ROS Nodes, ROS Messages, ROS Services, and ROS Actions. These components serve as essential building blocks for creating and implementing complex robot applications. Leveraging these ROS reusable components provides developers with benefits such as modularity, code reusability, and scalability throughout the robotics software development process. The ROS community also plays a vital role in supporting the utilization of these components. Developers can access numerous public code libraries and tools contributed by the ROS community, further facilitating component reuse. This rich ecosystem of resources provides a wide range of pre-existing functionalities, algorithms, and tools that developers can leverage to accelerate their development process and build more advanced robot applications.

ROS Packages [6], [10] serve as effective containers for organizing and distributing related functionalities within a robotics system. ROS Nodes [5], [29] are individual processes that perform specific tasks within a robot's system and can be reused in different robotics software to achieve similar functions. By leveraging ROS community-contributed messages [3], developers gain access to a wide range of functionalities and data formats, fostering collaboration and knowledge sharing. Reusing ROS services [4] enables developers to build modular and scalable robot applications, and

eventually leverage standardized interfaces for consistent and interoperable communication.

Overall, the utilization of ROS and its reusable components, along with the support from the ROS community, not only optimizes the reuse of ROS Nodes but also facilitates the development of robotics software.

### B. ROS Node Reuse Methods

Although reusing ROS Nodes is helpful in robot development, there is currently no dedicated method to directly address the issue of ROS Node reuse. Searching ROS-related code [16], [17], [32] is an important aspect of achieving ROS Node reuse, to use of existing resources and improve development efficiency.

Keyword search methods [20], [26]–[28] and fuzzy search methods are currently viable and directly applicable approaches to solving the problem of ROS Node reuse. Since ROS Node names typically follow a standardized naming structure, they may contain information related to the node type or functionality. By matching user-provided keywords using keyword search and fuzzy search methods, relevant nodes can be identified within the naming structure of ROS Nodes. In addition to keyword search and fuzzy search, subgraph matching methods and nlp search methods can assist in finding and reusing ROS Nodes. Subgraph matching methods [8], [9] utilize metadata and descriptive phrases in ROS Node documentation to search for desired ROS Nodes based on the user's query for subgraph structure. However, obtaining precise and detailed metadata and descriptive phrases from ROS Node documentation can be challenging, as this information may be scattered across different sources. NLP search methods [1], [2] involve parsing ROS Node description statements and calculating the similarity between the query statement and the node descriptions to perform the search. However, obtaining the textual descriptions of ROS Nodes may be difficult.

In summary, keyword search methods and fuzzy search methods are currently directly applicable to address the problem of searching for ROS Nodes based on user queries. Subgraph matching methods and nlp search methods may require more data and technical support to be implemented effectively in ROS Node searches.

## III. ROS NODE SEARCH METHOD

To address the challenge of scattered ROS Node knowledge, and the gap between the semantics of ROS knowledge and user queries, we propose a multi-feature semantic fusion method for searching ROS Nodes by utilizing a knowledge graph. Fig. 1 presents the overview of the ROS Node search method. This section first describes how we constructed the ROS domain dataset from ROS Wiki and GitHub. To identify the required knowledge for our ROS Node knowledge graph, we selected four user-concerned features as its entities by manually analyzing hundreds of user queries. Then we constructed our knowledge graph by performing entity recognition, entity typing and relation extraction. The similarities between entities and their corresponding descriptions in user queries were calculated to

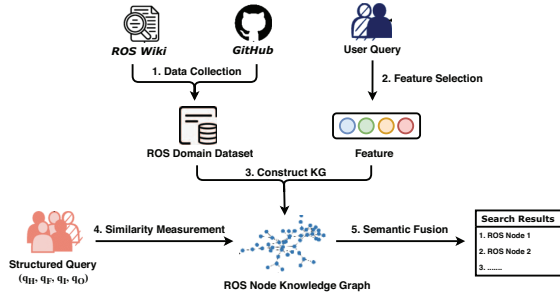


Fig. 1. The framework of the ROS Node search method.

bridge their semantic gap. Eventually, we proposed our fusion approach to support ROS Node search with the semantics of multiple entities in the knowledge graph.

#### A. Data Collection

Open-source software repositories are good data sources for characterizing the practices of robotics in the context of real-world projects [18], so we constructed a ROS dataset based on the steps in Fig. 2. The initial search (Step 1) is to collect robotics-based repositories in GitHub. Following that, in Step 2, we implemented a filtering process to exclude fork repositories, thereby mitigating duplication. In Step 3, we leveraged the "ROS" topics features on GitHub to identify repositories specifically related to ROS. As suggested in [13], we excluded repositories with less than 5 stars (Step 4) because they often indicate better quality and maintenance. We manually inspected the content of all the remaining repositories and determined that they were all related to ROS (Step 5). Since C++ and Python are the major programming languages to define ROS Nodes, we extracted them in Step 6 and 7. Next, in Step 8, we locally cloned 1,419 repositories and extracted 7,172 code files and related text descriptions. To complement the knowledge obtained from repositories, we also extracted textual descriptions of ROS Nodes and ROS packages from ROS Wiki, which helps enrich our dataset with ROS-related knowledge. By collecting scattered ROS Nodes' knowledge, we composed a comprehensive ROS dataset that contains knowledge on ROS-related practices for further analysis and exploration.

#### B. Feature Selection

To investigate the features that users are most interested in when querying ROS Nodes, we first randomly selected 100 queries related to ROS Nodes from ROS Answer<sup>1</sup> for analysis. ROS Answer is a community Q&A platform that contains questions and answers related to the open-source ROS. We manually extracted ROS-related features from these queries and gradually clarified the conceptual descriptions of these features within the queries. To further validate our hypothesis, we analyzed an additional 300 queries related to ROS Nodes from ROS Answer. We observed a gradual convergence of

<sup>1</sup><https://answers.ros.org/questions/>

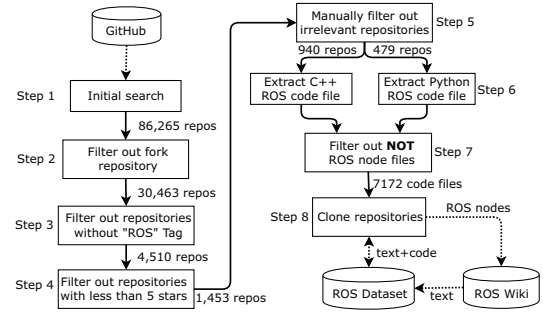


Fig. 2. Data collection and filtration process for our ROS domain dataset.

user-focused features, which are shown in TABLE I. This process represents a typical qualitative analysis approach [14]. Lastly, we identified *Function*, *Hardware*, *Input*, and *Output* as describing ROS Node features because they appeared in more than 20% of our selected queries. The definition of these ROS Nodes features is as follows:

- *Function*: the tasks or operations it can perform, e.g., object recognition, straight-line navigation, etc.
- *Hardware*: specific hardware devices which ROS Nodes may be associated with, for reading input, performing computations, or controlling external devices, e.g., IMU, Sensor, etc.
- *Input*: the data or messages it receives from other nodes or systems. Inputs can include sensor data, messages published by other ROS Nodes, or data read from external devices, e.g., Image, Pointcloud, etc.
- *Output*: the data or messages it publishes to other nodes or systems. Outputs can include processed data, actuator control commands, or information required by other ROS Nodes, e.g., visualization\_msgs::MarkerArray, etc.

The four features mentioned are not only more frequent in queries but also key to differentiate between different ROS Nodes. For example, if two nodes have the same *Function*, *Hardware*, and *Input*, but differ only in their *Output*, they can be considered as distinct. This distinction can assist ROS Node search systems in making accurate decisions when understanding the user's search intent.

#### C. ROS Node Knowledge Graph Construction

We organized and stored the scattered ROS Node knowledge by knowledge graph, focusing on the aforementioned query features as our target entities. Due to the diverse formats of entities within our corpus, we employed separate entity recognition approaches tailored to each specific entity. To support knowledge reasoning during ROS Node search, we performed entity typing for our proposed features. Finally, we further enriched our knowledge graph utilizing relation extraction techniques.

1) *Entity Recognition*: to bridge the semantic gap between descriptions in user queries and ROS domain knowledge, our ROS knowledge graph must include ROS Nodes, *Function*,

TABLE I  
THE PROPORTION OF DIFFERENT FUNCTIONALITY INDICATORS APPEARING IN QUERIES

| Indicator  | Functionality | Hardware Information | Input Data Format | Output Data Format | Algorithm |
|------------|---------------|----------------------|-------------------|--------------------|-----------|
| Proportion | 100%          | 57.14%               | 28.57%            | 35.71%             | 14.28%    |

TABLE II  
A TAXONOMY OF ROS NODE FUNCTIONS

| Category                | Subcategory                                                                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Hardware node           | Sensor node, Executor node, Interface node, Driver node                                                                                         |
| Data Processing node    | Registration node, Fusion node, Filter node, Assembler node, Combiner node, Cluster node, Convert node, Segment node, Publish node, Bridge node |
| Control node            | Motion control, Action control, Navigation control                                                                                              |
| Navigation node         | Navigation planning, Localization, Position estimation, Path Tracking and Control                                                               |
| Human-Robot Interaction | Speech interaction, Visual interaction, Display node, Feedback node, Tracking node                                                              |
| Analysis and Detection  | Anomaly Detection, Fault Analysis                                                                                                               |
| Simulation              | Physical simulation, Environmental simulation, Sensor simulation, Interactive simulation                                                        |

TABLE III  
A TAXONOMY OF ROS NODE HARDWARE

| Category | Subcategory                                                                                                                                                                   |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Robots   | Aerial, Component, Ground, Manipulator, Marine                                                                                                                                |
| Sensors  | 1D range finders, 2D range finders, 3D Sensors, Audio device, Cameras, Enviromental, Force Sensors, Motion Capture, Pose Estimation, Power Supply, RFID, IO Interfaces, Speed |
| Motor    | Motor Controller, Servo Controller                                                                                                                                            |

*Hardware*, *Input*, and *Output*. We employed distinct entity recognition approaches tailored to each specific entity type, as outlined below:

- *Input and Output*: we utilized a rule-based approach to identify ROS APIs and perform entity recognition from code files. We can identify relevant input and output entities associated with ROS Nodes by function calls such as `publish()`, `subscribe()`, `advertise()`, `call()`, etc.
- *Hardware*: we employed Part-of-Speech (POS) tagging [23] techniques to extract specific hardware entities that are described by proper noun phrases and adjective modifiers from the text. During the POS tagging process, we used tags such as <NNP>, <JJ>, <CD>, <NN>, to locate and extract hardware features related to ROS Nodes.
- *Function*: the first author and the third author read code files of 300 ROS Nodes and coded their functions. Then the two authors combined their obtained function codes and unified their expressions. For the disagreement and uncertainty on the ROS Node functionality definition, the two authors discussed and re-defined. We organized the functionality to generate a functionality taxonomy, which is shown in TABLE II.

2) *Entity Typing*: We have extracted relevant entities from scattered ROS Node data by entity recognition. However, the domain-specific descriptions of these entities (e.g., hardware: IFM O3M250 ToF camera) impede their matching with query

features. To enhance the usability and enrich the overall structure of the ROS Node knowledge graph, we completed the missing knowledge of our entities based on our predefined classification. For *Hardware*, we built its classification by referencing the standards on ROS Wiki page in TABLE III. A section titled "Hardware" provides information about various hardware devices supported by ROS. Leveraging the taxonomy, we linked the specific entities with their categories and subcategories. For example, "IFM O3M250 ToF camera" belongs to the category "Sensor/3D Sensors". Thus, we created an edge from "IFM O3M250 ToF camera" to "3D Sensor" and another from "3D Sensor" to "Sensor", which helps organize and represent hardware-related knowledge, making it easier to search and comprehend. Likewise, we performed similar completion for *Function* utilizing our taxonomy in TABLE II. Therefore, we can better describe and organize the functionality of ROS Nodes, assisting users in filtering nodes based on their task requirements. For *Input* and *Output*, they can take the form of topics or services in ROS. A ROS Node can send or receive data by publishing or subscribing topics. Each topic transfers a specific type of message, thus we added a message entity to enrich the input and output semantics of ROS Nodes. As for services, they send requests and collect responses between ROS Nodes, providing required inputs and outputs. By adding the message and service types, we associated input-output information with specific ROS Nodes, which can clearly represent the relationship between them and help users understand input-output information more easily. For example, by associating the topic "red\_light\_intensity/commanded" with the message entity "std\_msgs::Bool", we can know that the ROS Node requires a boolean value as its input or output.

By adopting the entity typing technique, our knowledge graph provides a more comprehensive representation of ROS Nodes and decreases the semantic gap between the user query and ROS domain knowledge.



3) *Relation Extraction*: In traditional KG, a fact is represented by a triplet, which refers to subject, predicate and object (SPO). Relation extraction aims to construct the triplet of two entities and their relation. In the ROS domain, ROS Node is the central entity, hence only the relationships between ROS Nodes and other types of entities are needed. And, 9 relations are defined in this paper. They are *HasTopic*, *HasService*, *HasPackage*, *HasMessage*, *HasServiceType*, *HasCategory*, *HasHardware*, *HasFunction* and *HasCodeURL*.

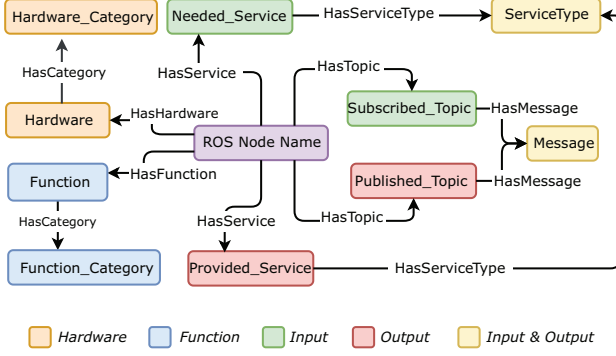


Fig. 3. The pattern diagram of the ROS Nodes knowledge graph.

By following the three steps outlined above, we have successfully created a ROS Node domain knowledge graph which can provide support for the reusability of the ROS Nodes and ROS software development. The schema of the knowledge graph is displayed in Fig. 3, and the basic statistics of ROS Node knowledge graph are illustrated in TABLE IV. The constructed ROS Node knowledge graph consists of 12,350 entities and 36,029 triples. Considering that some ROS Nodes may not have inputs and outputs with regard to services, each ROS Node is connected to 5.17 (<6) entities. Due to the inclusion of hardware and function knowledge using entity typing, the average path length from a ROS Node to hardware and function is 2.97 and 2, respectively.

TABLE IV  
BASIC STATISTICS OF THE EXTRACTED ROS NODES KNOWLEDGE GRAPH

| # Entities | # Triples | Avg. Deg. of ROS Node | Avg. Path Len of Hardware | Avg. Path Len of Function |
|------------|-----------|-----------------------|---------------------------|---------------------------|
| 12,350     | 36,029    | 5.17                  | 2.97                      | 2                         |

#### D. Semantic Similarity Calculation

To eliminate the semantic gap between the descriptions of features mentioned in user queries and the entities in the knowledge graph, we mapped corresponding features and performed semantic similarity calculation utilizing our constructed knowledge graph and a pre-trained language model. As features are represented differently in our knowledge graph, we calculated their similarity as follows:

1) *Hardware Similarity*: Given the head entity  $n_0$  is the ROS Node entity,  $h_1$  is a Hardware entity, and  $h_2, \dots, h_k$  is a series of Hardware\_Category entity. If there exist  $r_0 = \text{HasHardware}$  and  $r_i = \text{HasCategory}, i \in [1, k-1]$ , we have a valid hardware searching path:

$$\text{hardware} = \{n \xrightarrow{r_0} h_1 \xrightarrow{r_1} h_2 \dots h_{k-1} \xrightarrow{r_{k-1}} h_k\} \quad (1)$$

If the description of *Hardware* in the query exactly matches the Hardware entity  $h_1$ , the similarity is set to 1. Otherwise, we leverage our knowledge graph to complete the calculation. Note that entities  $h_2 \dots h_{k-1}$  are the parent hardware categories, their hierarchies increase but associations get weaker when the path gets longer. Therefore, we calculate the hardware similarity using a decay weighted sum of all similarity values of the entities on the hardware searching path, which is shown in Equation (2) and Fig. 4.

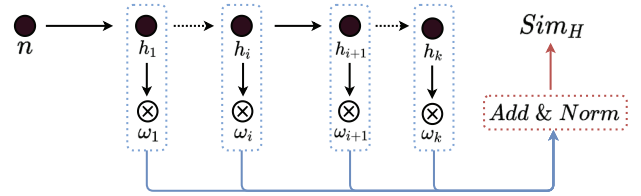


Fig. 4. The process of calculating hardware feature if the description of hardware in the query is not exactly match.

$$\text{Sim}_H(q_H, n) = \begin{cases} 1, & \text{if } q_H = h_1 \\ \frac{\sum_i w_i \cdot \text{Sim}_h(q_H, h_i)}{\sum_k w_i}, & \text{otherwise.} \end{cases} \quad (2)$$

where  $w_i = 1/(2^{i-1})$ ,  $i = 1 \dots k$ , and  $\text{Sim}_h(q_H, h_i)$  is the similarity between  $q_H$  and  $h_i$ .

As for  $\text{Sim}_h(q_H, h_i)$ , we calculate it using the cosine similarity of Word2Vec [21] if  $q_H$  is a common word. Otherwise, we use edit distance to calculate the similarity of  $q_H$  and  $h_i$ , as shown in Equation (3).

$$\text{Sim}_h(q_H, h_i) = \begin{cases} \cos(V_{q_H}, V_{h_i}), & \text{if } q_H \in W_c \\ \text{Levenshtein}(q_H, h_i), & \text{otherwise.} \end{cases} \quad (3)$$

where  $W_c$  is the set of common words obtained from WordNet [22].  $V_{q_H}$  and  $V_{h_i}$  are the vectors calculated by Word2Vec.

Therefore, we can adequately meet the task requirements for analyzing user-provided hardware keywords.

2) *Function Similarity*: In our knowledge graph, each ROS Node only has one Function entity and one Function\_Category entity, whose descriptions are  $f$  and  $f_c$  respectively. We use a pre-trained Sentence-BERT model [25] in Equation (4) to calculate their function similarity  $\text{Sim}_f(q_F, f)$  with the functional description  $q_F$  in the user query, because it has better performance in search scenarios and suitability for the task requirements. However, Sentence-BERT takes sentences

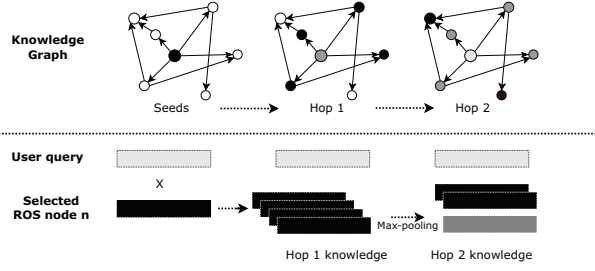


Fig. 5. The framework of calculating Input or Output Similarity. It tasks user query's input/output feature as input, and outputs the most similarity probability.

as its input. We utilize the text descriptions of *Function* in our dataset to convert our entities to sentences.

$$Sim_f(q_F, f) = \cos(S\_BERT(q_F), S\_BERT(f)) \quad (4)$$

$$Sim_F(q_F, n) = \max(Sim_f(q_F, f), Sim_f(q_F, fc)) \quad (5)$$

Then we calculate the cosine similarity between the vector representations of our entity sentences and  $q_F$ . The bigger one is taken as our function similarity  $Sim_F(q_F, n)$  between ROS Node  $n$  and  $q_F$ , as shown in Equation (5).

3) *Input/Output Similarity*: *Input* can be represented by *Subscribed\_Topic* and *Needed\_Service* entities and *Output* can be represented by *Published\_Topic* and *Provided\_Service*. However, considering only the names of topics and services is not sufficient to fully characterize the input and output information of ROS Nodes. Detailed input and output information of ROS Nodes is stored in the message types corresponding to topics and services. Therefore, when analyzing the user's input and output information, we focus on the services, topics, and on the message types associated with them.

The process of calculating input and output similarity is depicted in Fig. 5 and Equations (6)-(9). Initially, for a candidate ROS Node entity  $n$ , we first retrieve all the input/output entities ( $I_i/O_i$ ) within one hop  $H_1(n)$ , i.e., all related services and topics, and calculate their similarity corresponding to the input/output description  $q_{I/O}$  in the user query. The maximum similarity  $Sim_{H_1}(q_{I/O}, n)$  is stored and its one-hop node  $I_m/O_m$  must be further explored to fully represent the input and output information. Hence, we extract its two-hop entities, i.e., messages and service types from the knowledge graph and calculate their similarity to  $q_{I/O}$  again. The maximum similarity among the stored one-hop similarity  $Sim_{H_1}$  and calculated two-hop similarity  $Sim_{H_2}$  is the final input/output similarity for the ROS Node. All similarity is calculated by the cosine similarity of the vector representation of Word2Vec. We can determine to what extent the input/output information of the candidate ROS Node meets the requirements of the user query.

$$Sim_{H_1}(q_{I/O}, n) = \max_{I_i/O_i \in H_1(n)} (\cos(V_{q_{I/O}}, V_{I_i/O_i})) \quad (6)$$

$$I_m/O_m = \arg \max_{I_i/O_i \in H_1(n)} (\cos(V_{q_{I/O}}, V_{I_i/O_i})) \quad (7)$$

$$Sim_{H_2}(q_{I/O}, n) = \max_{\substack{I_i/O_i \in H_2(n) \\ \cap H_1(I_m/O_m)}} (\cos(V_{q_{I/O}}, V_{I_i/O_i})) \quad (8)$$

$$Sim_{I/O}(q_{I/O}, n) = \max(Sim_{H_1}, Sim_{H_2}) \quad (9)$$

where  $H_k(n)$  represents the set of all  $k$ -hop entities of  $n$ .

#### E. Multi-feature Semantic Fusion Model

After calculating the semantic similarity of different features in user queries and knowledge graph, we integrate them to support ROS Node search. To achieve this goal, we leveraged a linear regression model to weight and combine different features, which is shown in Eq (10).

$$Sim_{node}(q, n) = \frac{\sum_i w_i \cdot Sim_i(q_i, n)}{\sum_i w_i} \quad (10)$$

where  $i \in \{H, F, I, O\}$ . The capital letters in the set represent *hardware*, *function*, *input* and *output* respectively. However, as our analysis on user queries suggests, users may search ROS Nodes without all features in their queries, only the included ones are integrated into the linear model. By training on a labeled dataset, the linear model can integrate the features by learning the weights between different features. During the application phase, our model can calculate and rank the similarity between the user query and all ROS Nodes based on the model's output to find the ROS Nodes that best match the user query, considering the combined influence of multiple features.

#### IV. EXPERIMENTAL SETTINGS

We present our experiments and the corresponding results including dataset, queries and comparison models.

##### A. Query Dataset

For our training dataset, we carefully selected 250 queries from ROS Answer that included all four features. The selection facilitate effective learning of the relationships between all features and their corresponding weights. It can also enhance model's generalization capability when certain features are absent from user queries.

To conduct a comprehensive evaluation of the model's performance in real-world scenarios, we randomly collected a test dataset comprising 100 queries. TABLE V illustrates the coverage of features within this dataset. It includes 20 single-feature queries, 40 double-feature queries, 30 triple-feature queries and 10 quadruple-feature queries. For instance, the query of ROS Answer 415723 "Merging and Tracking Obstacles from Multiple Robots" is a double-feature query, whose features include *Hardware* ("Multiple Robots") and *Function* ("Merging and Tracking Obstacles"). By employing this test dataset, we assess its effectiveness in utilizing available features while minimizing reliance on missing ones. This

TABLE V  
AN OVERVIEW OF THE TEST DATASET COVERAGE OF FOUR FEATURES

| # of features | Single | Double |    |   | Triple |    |   | Quadruple |
|---------------|--------|--------|----|---|--------|----|---|-----------|
| Hardware      |        | ✓      |    |   | ✓      |    | ✓ | ✓         |
| Function      | ✓      | ✓      | ✓  | ✓ | ✓      | ✓  | ✓ | ✓         |
| Input         |        |        | ✓  |   | ✓      | ✓  |   | ✓         |
| Output        |        |        |    | ✓ |        | ✓  | ✓ | ✓         |
| Sum           | 20     | 22     | 15 | 3 | 16     | 11 | 3 | 10        |

evaluation enables us to examine the model’s robustness and its capacity to handle real-world queries where features may be absent.

We trained our linear model on the training dataset and performed regression analysis on it. Its search performance and comparisons to other approaches were conducted on the test dataset.

### B. Baselines

Due to the current absence of effective ROS Node search methods, we choose four search methods for comparison to validate the effectiveness of our proposed approach, including Keyword-based Search, Fuzzy Search, Subgraph Matching, and nlp search methods. By comparing the search results obtained from these different methods, we evaluate the performance of our proposed ROS Node search method.

- **Keyword-based Search:** TF-IDF (Term Frequency-Inverse Document Frequency) [19] is a statistical method to evaluate the importance of words in a text, and applied in the field of information retrieval. By using the TfidfVectorizer from the scikit-learn library, we can perform feature extraction and vectorization of the ROS Nodes corpus for the calculation of similarity with user queries and the retrieval of the most relevant ROS Nodes based on user queries.
- **Fuzzy Search:** While ROS Documentation<sup>2</sup> is not specifically designed as a Website to search for ROS Nodes, it is possible to find the desired ROS Nodes by performing fuzzy matching on a large number of ROS Document names. Despite its limitations in searching for ROS Nodes, this method is an effective approach provided by the official ROS resources. By applying fuzzy matching to the keywords in a user query, we can identify the relevant ROS Nodes. Therefore, we also include ROS Documentation in this comparison.
- **Subgraph Matching:** VF2++ [12] is an efficient subgraph matching method, and its application to the matching process of ROS Node knowledge graphs can enable faster identification of ROS Nodes that meet user requirements. Firstly, by building a target subgraph based on user queries and matching it with the ROS Nodes in the knowledge graph, subgraphs that resemble the target subgraph

<sup>2</sup><https://docs.ros.org/en>

can be found. These subgraphs represent combinations of nodes that fulfill specific task requirements. The use of a subgraph-based matching method helps capture the semantic information of ROS Nodes, allowing for more precise node selections for users.

- **NLP search methods:** SentenceBERT [25] is a deep learning model specifically designed to handle the problem of embedding natural language text. In the ROS Node search task, it is possible to calculate the similarity between user queries and ROS Node description sentences. By using the SentenceTransformer model, the user query and node description sentences can be transformed into semantic vector representations, allowing us to retrieve ROS Nodes that are close to the query in the vector space.

### C. Evaluation Metrics

We evaluate the goodness-of-fit of our linear model by considering Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared, which indicates the effectiveness of our integration of ROS Node knowledge. The effects of our features are measured using the ANOVA Type-II test applied to the linear regression model. Additionally, to measure the effectiveness of the ROS Node search method, we primarily focus on the Normalized Discounted Cumulative Gain (NDCG) metric [30]. It is a widely-used metric in information retrieval and recommendation systems to assess the effectiveness of ranking algorithms. It measures the quality of ranked lists by considering both relevance and position of the items, which assesses the effectiveness of algorithms in providing relevant and well-ordered recommendations or search results. Specifically, we use  $NDCG@ \{1,5,10,20\}$  in our experiments.

## V. RESULTS

We conducted four sets of experiments to evaluate various aspects of our study. Firstly, we assessed the performance of ROS Node knowledge fusion using regression analysis. Secondly, we measured the performance of ROS Node search methods. Thirdly, we examined the effectiveness of multi-feature settings in ROS Node search methods. Lastly, we evaluated the impact of parameter settings.

### A. Regression Analysis

We evaluated the feasibility of employing a linear model to fuse multiple features of ROS Nodes. The evaluation results, displayed in TABLE VI, were obtained by using multi-features as independent variables and fitting their weights using our training dataset.

Table VI presents the regression results of our linear model, demonstrating its low MSE and RMSE of 0.0483 and 0.2198 respectively. The R-squared value is 0.6753, which indicates a high goodness-of-fit of the model. Additionally, as shown by the results of ANOVA Type-II test, all four features significantly influenced our similarity integration. This finding supports the effectiveness of employing a linear model to fuse multiple ROS Node features.

TABLE VI  
THE RESULTS OF LINEAR REGRESSION

| Evaluation Metrics |  | MSE    | RMSE   | R-squared |
|--------------------|--|--------|--------|-----------|
| Value              |  | 0.0483 | 0.2198 | 0.6753    |

| Multi-feature | Function       | Hardware       | Input          | Output         |
|---------------|----------------|----------------|----------------|----------------|
| Pr(>F)        | $7.531e^{-47}$ | $1.233e^{-13}$ | $1.644e^{-05}$ | $4.376e^{-07}$ |

### B. Searching Performance

The MUSE searching accuracy of ROS Nodes on our queries is shown in TABLE VII. For each search type, we have selected a baseline method that is suitable for ROS Nodes. Baseline methods' relative performances comparing with MUSE are shown in percentages.

MUSE demonstrates a significant improvement over the baseline methods, achieving a minimum 7% increase across all evaluation metrics. Notably, in terms of earlier positions (NDCG@1), which hold greater importance for user satisfaction, MUSE exhibits even stronger performance. Specifically, there are at least 20% and 5% enhancement in NDCG@1 and NDCG@5 compared to NDCG@20. The reason for MUSE's success in positioning answers closely related to the user query at the top positions is attributed to its ability to leverage the advantages of semantic calculations using multi-features (*Function*, *Hardware*, *Input*, and *Output*). This approach allows for more precise and accurate searching of the ROS Nodes that align with the users' requirements.

MUSE retains the advantages of the baseline approach in the task of searching ROS Nodes. Firstly, MUSE preserves keywords in the search task by using a knowledge graph to store important multi-feature keywords from the ROS corpus. To leverage the advantages of graph matching in knowledge graph-based ROS Node search, we use the edges of the knowledge graph as constraints for calculating similarity. To provide users with a better search experience, we utilize entity typing to integrate feature information from the knowledge graph. Lastly, to reduce the semantic gap between user descriptions and ROS domain knowledge, we employ a pre-trained model to compute similarity between different features.

This experiment demonstrates MUSE's ability to improve the accuracy of ROS Node search tasks. MUSE's advantages also favor ROS Node search engines' user satisfaction: More improvements are at early ranking positions.

### C. Effectiveness of ROS Node Features

In this experiment, the same ROS Node searching model was employed, but different user query keywords were utilized. The evaluation dataset was divided based on the multi-features (*Hardware*, *Function*, *Input*, *Output*), and the effectiveness of each feature in searching ROS Nodes was

evaluated. The results obtained from different multi-feature queries are presented in TABLE VIII.

TABLE VIII reveals that the *Function* feature has the most significant impact on ROS Node search tasks, followed by *Hardware*. Conversely, the impact of *Input* and *Output* features on search tasks is comparatively smaller. The *Function* feature carries a close to 50% impact on ROS Node search tasks, as ROS Nodes are primarily intended to provide specific functionalities for robot software. If users do not specify the desired function, it becomes challenging to find ROS Nodes that meet the requirements. The *Hardware* feature plays a crucial role in determining the suitability of ROS Nodes for specific hardware requirements. It becomes especially important in ROS Node search tasks that involve specialized hardware, as it enables a quick identification of suitable nodes. On the other hand, the *Input* and *Output* features complement the *Function* feature and play a significant role in finding nodes related to data processing.

This experiment can help us understand the importance of the proposed features (*Function*, *Hardware*, *Input*, *Output*) in ROS Node search tasks. By mapping the acquired query features to the entities in the ROS Node knowledge graph, we can assist users in finding the desired ROS Node.

### D. Effectiveness of MUSE Parameters

To minimize the semantic gap between user queries and the ROS Node knowledge graph, MUSE utilizes linear regression to combine the contributions of different features in the ROS Node search task. In the fourth experiment, the effects of parameter settings on feature fusion are explored by contrasting the weights of multiple features set as an average score with the weights obtained from training a linear model. Setting the weights of different features as the same assumes equal importance for all features in the search task. However, training a linear model enables the weights to be learned from the data, taking into account the actual contribution of each feature to the search task.

TABLE IX displays the results regarding the effectiveness of parameter settings on multi-feature fusion. Relative performance is compared with the corresponding trained parameters. The results demonstrate the effectiveness of trained parameters settings on searching ROS Nodes based on user query. Setting the parameters as the mean value may lose the multiple features and reduce the information carried by the query. Therefore, parameters trained based on a large number of user queries would more accurately meet the requirements of ROS Node search tasks. Also, the exact match information is not preserved in the mean parameters when there are multiple query entities.

The results indicate that determining the parameters for multi-feature fusion is a challenging task. It requires finding the appropriate parameter settings that achieve a balance in the fusion of multiple features. While there are more complex abstraction strategies available, we believe that the linear regression model strikes the right balance between model



TABLE VII  
COMPARISON OF DIFFERENT ROS NODE SEARCH METHODS

| Method Classification    | Method       | NDCG@1        |              | NDCG@5        |              | NDCG@10       |              | NDCG@20       |              |
|--------------------------|--------------|---------------|--------------|---------------|--------------|---------------|--------------|---------------|--------------|
|                          |              | accuracy      | performances | accuracy      | performances | accuracy      | performances | accuracy      | performances |
| <b>Keyword-based</b>     | TF-IDF       | 0.3377        | -59.69%      | 0.5746        | -43.37%      | 0.6163        | -34.38%      | 0.7571        | -8.90%       |
| <b>Subgraph Matching</b> | VF2++        | 0.2575        | -66.15%      | 0.4112        | -47.62%      | 0.4615        | -43.33%      | 0.7084        | -14.76%      |
| <b>Fuzzy Search</b>      | ROS Doc      | 0.4562        | -38.23%      | 0.6738        | -14.17%      | 0.6846        | -15.93%      | 0.7503        | -9.72%       |
| <b>NLP Search</b>        | SentenceBert | 0.5458        | -26.10%      | 0.68077       | -13.28%      | 0.7350        | -9.74%       | 0.7720        | -7.11%       |
| -                        | <b>MUSE</b>  | <b>0.7386</b> |              | <b>0.7851</b> |              | <b>0.8144</b> |              | <b>0.8311</b> |              |

TABLE VIII  
THE EFFECTIVENESS OF ROS NODES MULTI-FEATURES

| Query Selection     | NDCG@1   |             | NDCG@5   |             | NDCG@10  |             | NDCG@20  |             |
|---------------------|----------|-------------|----------|-------------|----------|-------------|----------|-------------|
|                     | accuracy | performance | accuracy | performance | accuracy | performance | accuracy | performance |
| <b>w/ Hardware</b>  | 0.8648   | 34.28%      | 0.8848   | 24.5%       | 0.8672   | 11.87%      | 0.8814   | 12.75%      |
| <b>w/o Hardware</b> | 0.6440   |             | 0.7103   |             | 0.7642   |             | 0.7690   |             |
| <b>w/ Function</b>  | 0.7490   | 56.64%      | 0.8220   | 55.10%      | 0.8455   | 46.50%      | 0.847775 | 44.07%      |
| <b>w/o Function</b> | 0.3247   |             | 0.3690   |             | 0.4522   |             | 0.4741   |             |
| <b>w/ Input</b>     | 0.7655   | 8.93%       | 0.7995   | 4.38%       | 0.8404   | 7.44%       | 0.8419   | 8.30%       |
| <b>w/o Input</b>    | 0.7027   |             | 0.7659   |             | 0.7778   |             | 0.7720   |             |
| <b>w/ Output</b>    | 0.7777   | 6.19%       | 0.7960   | 0.13%       | 0.8267   | 0.85%       | 0.8312   | 2.70%       |
| <b>w/o Output</b>   | 0.7323   |             | 0.7949   |             | 0.8197   |             | 0.8088   |             |

TABLE IX  
THE EFFECTIVE OF PARAMETER SETTINGS ON MULTI-FEATURE FUSION.

| Parameter Setting    | NDCG@1   |               | NDCG@5   |               | NDCG@10  |               | NDCG@20  |               |
|----------------------|----------|---------------|----------|---------------|----------|---------------|----------|---------------|
|                      | accuracy | comparsion    | accuracy | comparsion    | accuracy | comparsion    | accuracy | comparsion    |
| Avg-Linear parameter | 0.7265   | <b>+1.64%</b> | 0.7658   | <b>+2.45%</b> | 0.7797   | <b>+4.26%</b> | 0.7706   | <b>+4.98%</b> |
| Linear parameter     | 0.7386   |               | 0.7851   |               | 0.8144   |               | 0.8311   |               |

complexity and multi-feature fusion, considering the size of our training data.

## VI. CONCLUSION AND FUTURE WORK

Through an analysis of existing ROS Node search tools and a study of users' requirements, the current tools fail to adequately meet users' needs when it comes to ROS Node queries. To address these shortcomings and fulfill the users' demands for ROS Node search, we constructed a comprehensive knowledge graph that encompasses ROS Node knowledge. This knowledge graph encompasses various aspects of ROS Nodes, including functional descriptions, supported hardware devices, input requirements, and output results. By constructing this

knowledge graph, we offer users comprehensive and precise information about ROS Nodes that align with their query requirements. Leveraging the knowledge graph as a foundation, we enhanced and integrated existing ROS Node search methods to develop a more tailored and suitable ROS Node search method specifically designed for the ROS domain. In this search method, we minimize the differences between ROS domain text and natural language text, to obtain the desired ROS Nodes without the need for domain-specific terminology. Additionally, we incorporated the "Function," "Hardware," "Input," and "Output" information to facilitate ROS Node search tasks, improving the accuracy of the search method.

Through comparative experiments with existing ROS Node

search methods, Our approach demonstrated superior performance. Our approach effectively integrates the strengths of various methods, including keyword-based querying, subgraph matching, and fuzzy search. By leveraging the unique characteristics of ROS Nodes, our approach achieves higher accuracy in searching for ROS Nodes. In addition, we conducted an evaluation to assess the impact of the "Function," "Hardware," "Input," and "Output" features on the performance of MUSE. The results confirmed the significance of these four features in effectively describing and recommending ROS Nodes. This analysis highlights the necessity of incorporating these features in the search process. Furthermore, we examined the influence of parameters on the performance of the ROS Node search method. The findings revealed that optimal parameter settings yield superior performance compared to average parameter settings.

Our work will further focus on code reuse in the ROS Node domain. Leveraging the ROS Node knowledge graph and the search method, we aim to provide support for developing robot software by delving into extensive user data.

## REFERENCES

- [1] Ivar Örn Arnarsson, Otto Frost, Emil Gustavsson, Mats Jirstrand, and Johan Malmqvist. Natural language processing methods for knowledge management—applying document clustering for fast search and grouping of engineering documents. *Concurrent Engineering*, 29(2):142–152, 2021.
- [2] Illia Balush, Victoria Vysotska, and Solomiia Albota. Recommendation system development based on intelligent search, nlp and machine learning methods. In *MoMLeT+ DS*, pages 584–617, 2021.
- [3] Sun Bo, Xinjun Mao, Shuo Yang, and Long Chen. Towards an efficient searching approach of ros message by knowledge graph. In *2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC)*, pages 934–943. IEEE, 2022.
- [4] Radhia Bouziane, Labib Sadek Terrissa, and Soheib Ayad. Semantic web services for ros: a robot as a service approach. *Automated Software Engineering*, 29(2):49, 2022.
- [5] Darko Bozhinski, Esther Aguado, Mario Garzon Oviedo, Carlos Hernandez, Ricardo Sanz, and Andrzej Wąsowski. A modeling tool for reconfigurable skills in ros. In *2021 IEEE/ACM 3rd International Workshop on Robotics Software Engineering (RoSE)*, pages 25–28. IEEE, 2021.
- [6] Long Chen, Xinjun Mao, Yinyuan Zhang, Shuo Yang, and Shuo Wang. An efficient ros package searching approach powered by knowledge graph. In *Proceedings of the International Conference on Software Engineering and Knowledge Engineering*, pages 411–416, 2021.
- [7] Yi Chen, Wei Wang, Ziyang Liu, and Xuemin Lin. Keyword search on structured and semi-structured data. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*, pages 1005–1010, 2009.
- [8] Yuhui Ding, Quanming Yao, Huan Zhao, and Tong Zhang. Diffmg: Differentiable meta graph search for heterogeneous graph neural networks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 279–288, 2021.
- [9] Yuan Fang, Wenqing Lin, Vincent W Zheng, Min Wu, Kevin Chen-Chuan Chang, and Xiao-Li Li. Semantic proximity search on graphs with metagraph-based learning. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*, pages 277–288. IEEE, 2016.
- [10] Stephen Hart, Paul Dinh, and Kimberly Hambuchen. The affordance template ros package for robot task programming. In *2015 IEEE international conference on robotics and automation (ICRA)*, pages 6227–6234. IEEE, 2015.
- [11] Yingbing Hua, Stefan Zander, Mirko Bordignon, and Björn Hein. From automationml to ros: A model-driven approach for software engineering of industrial robotics using ontological reasoning. In *2016 IEEE 21st International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1–8. IEEE, 2016.
- [12] Alpár Jüttner and Péter Madarasi. Vf2++—an improved subgraph isomorphism algorithm. *Discrete Applied Mathematics*, 242:69–81, 2018.
- [13] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M German, and Daniela Damian. An in-depth study of the promises and perils of mining github. *Empirical Software Engineering*, 21:2035–2071, 2016.
- [14] BRUCE L BERG. Qualitative research methods for the social sciences. 2001.
- [15] Wangchao Le, Feifei Li, Anastasios Kementsietsidis, and Songyun Duan. Scalable keyword search on large rdf data. *IEEE Transactions on knowledge and data engineering*, 26(11):2774–2788, 2014.
- [16] Mingwei Liu, Xin Peng, Qingtao Jiang, Andrian Marcus, Junwen Yang, and Wenyun Zhao. Searching stackoverflow questions with multi-faceted categorization. In *Proceedings of the 10th Asia-Pacific Symposium on Internetware*, pages 1–10, 2018.
- [17] Kien Luong, Mohammad Hadi, Ferdian Thung, Fatemeh Fard, and David Lo. Facos: Finding api relevant contents on stack overflow with semantic and syntactic analysis. *arXiv preprint arXiv:2111.07238*, 2021.
- [18] Ivano Malavolta, Grace Lewis, Bradley Schmerl, Patricia Lago, and David Garlan. How do you architect your robots? state of the practice and guidelines for ros-based systems. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice*, pages 31–40, 2020.
- [19] Justin Martineau and Tim Finin. Delta tfidf: An improved feature space for sentiment analysis. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 3, pages 258–261, 2009.
- [20] Yinbin Miao, Wei Zheng, Xiaohua Jia, Ximeng Liu, Kim-Kwang Raymond Choo, and Robert H Deng. Ranked keyword search over encrypted cloud data through machine learning method. *IEEE Transactions on Services Computing*, 16(1):525–536, 2022.
- [21] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [22] George A. Miller. Wordnet: A lexical database for english. *Commun. ACM*, 38(11):39–41, nov 1995.
- [23] Idza Aisara Norabid and Fariza Fauzi. Rule-based text extraction for multimodal knowledge graph. *International Journal of Advanced Computer Science and Applications*, 13(5), 2022.
- [24] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, Andrew Y Ng, et al. Ros: an open-source robot operating system. In *ICRA workshop on open source software*, volume 3, page 5. Kobe, Japan, 2009.
- [25] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.
- [26] Charlotte Rudnik, Thibault Ehrhart, Olivier Ferret, Denis Teyssou, Raphaël Troncy, and Xavier Tannier. Searching news articles using an event knowledge graph leveraged by wikidata. In *Companion proceedings of the 2019 world wide web conference*, pages 1232–1239, 2019.
- [27] Yuxuan Shi, Gong Cheng, Trung-Kien Tran, Evgeny Kharlamov, and Yulin Shen. Efficient computation of semantically cohesive subgraphs for keyword-based knowledge graph exploration. In *Proceedings of the Web Conference 2021*, pages 1410–1421, 2021.
- [28] Yuxuan Shi, Gong Cheng, Trung-Kien Tran, Jie Tang, and Evgeny Kharlamov. Keyword-based knowledge graph exploration based on quadratic group steiner trees. In *IJCAI*, volume 2021, pages 1555–1562, 2021.
- [29] Christopher S Timperley, Tobias Dürschmid, Bradley Schmerl, David Garlan, and Claire Le Goues. Rosdiscover: Statically detecting runtime architecture misconfigurations in robotics systems. In *2022 IEEE 19th International Conference on Software Architecture (ICSA)*, pages 112–123. IEEE, 2022.
- [30] Yining Wang, Liwei Wang, Yuanzhi Li, Di He, and Tie-Yan Liu. A theoretical analysis of ndcg type ranking measures. In *Conference on learning theory*, pages 25–54. PMLR, 2013.
- [31] Thomas Witte and Matthias Tichy. Inferred interactive controls through provenance tracking of ros message data. In *2021 IEEE/ACM 3rd International Workshop on Robotics Software Engineering (RoSE)*, pages 67–74. IEEE, 2021.
- [32] Wenyuan Xu, Xiaobing Sun, Jiajun Hu, and Bin Li. Repersp: recommending personalized software projects on github. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 648–652. IEEE, 2017.